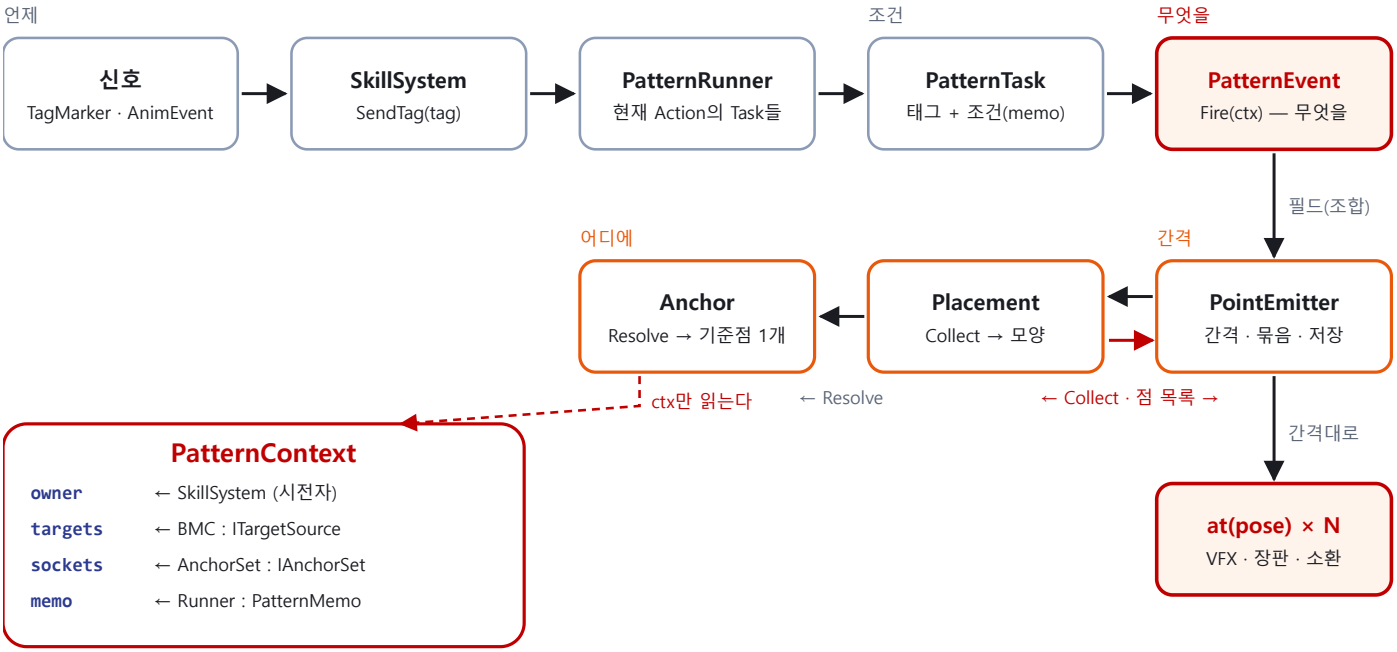
Pattern Event 구조 — 언제 · 무엇을 · 어디에 · 간격

신호(태그)가 오면 Task가 **태그 + 조건**으로 거르고, Event가 **무엇을** 한다. 점이 필요한 Event는 **PointEmitter**를 필드로 들고 → Placement가 **어디에** → Emitter가 **간격**대로 돌린다. 아래층은 전부 **PatternContext**만 읽는다.

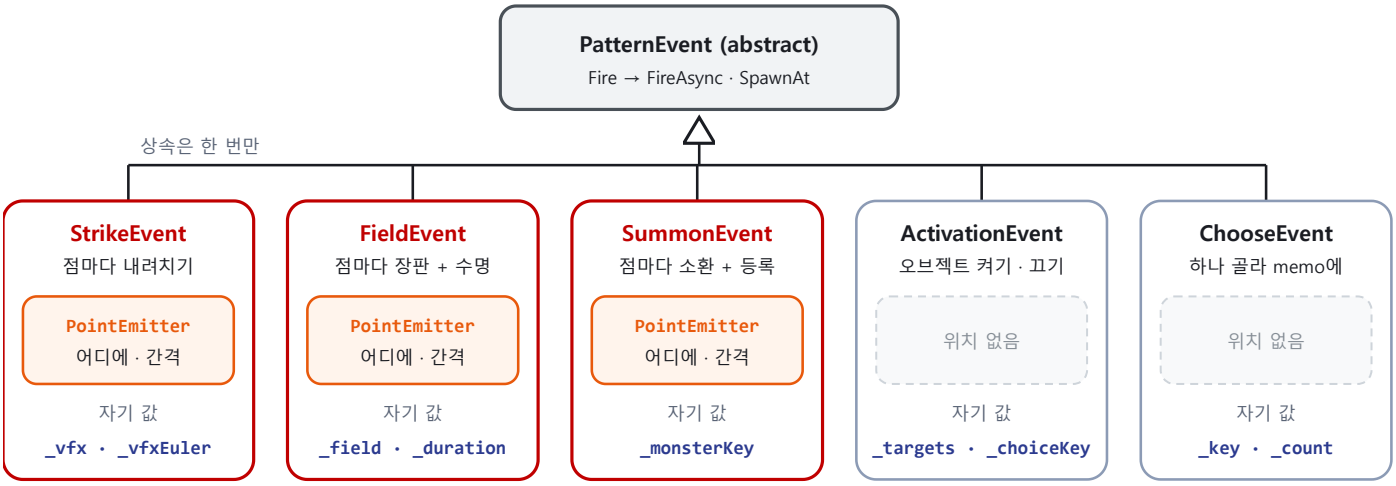
1. 전체 흐름



층마다 하나씩만 안다

1. **언제 — 신호 · Task**
   - 타임라인 TagMarker가 태그를 보내고, 현재 Action의 Task 중 태그와 조건(memo 값)이 맞는 것만 Fire
2. **무엇을 — Event**
   - PatternEvent를 한 번만 상속. "점 하나에 무엇을 하나"만 쓴다
3. **어디에 · 간격 — PointEmitter · Placement · Anchor**
   - 상속이 아니라 필드로 조합 — 다섯 Event가 같은 부품을 쓴다

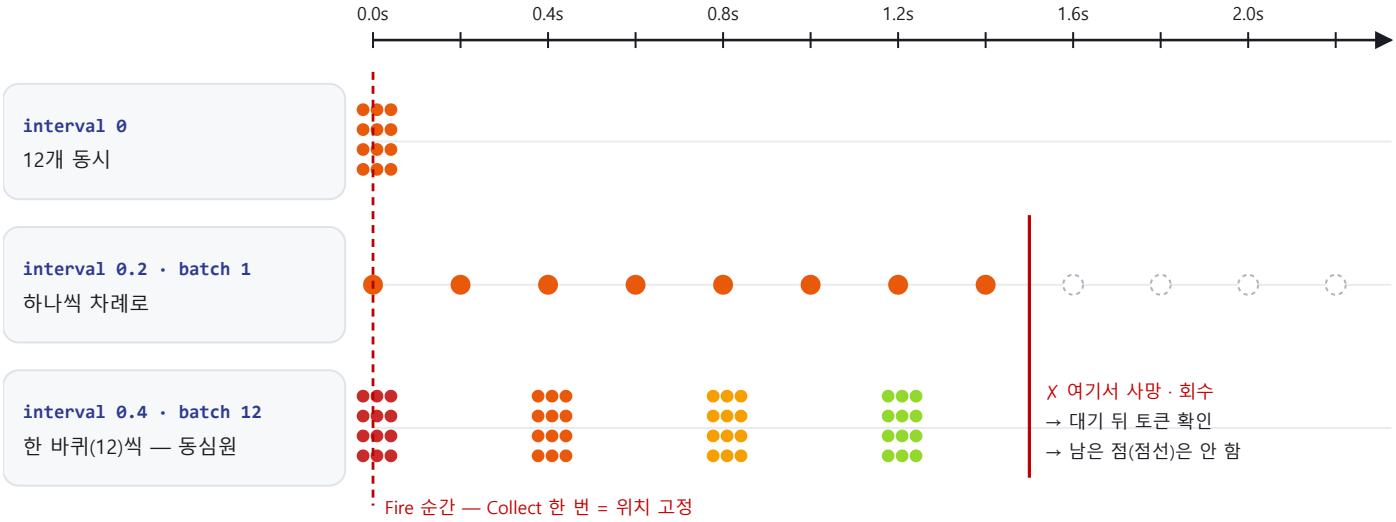
2. Event — 상속은 한 번, 위치는 조합



"한 번 때리기"와 "동심원 48번 때리기"는 같은 StrikeEvent — PointEmitter 설정만 다르다

3. PointEmitter — 간격 · 묶음

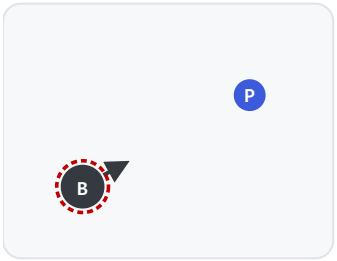
점 12개를 **어떤 리듬으로** 쓰느냐는 `interval`과 `batch` 두 값이 정한다.



4. Placement — 기준점 4가지 × 모양 4가지

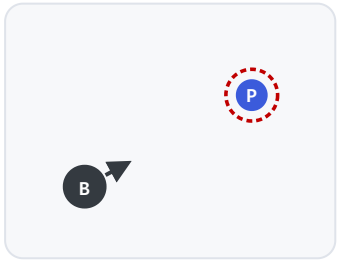
기준점 (Anchor.source)

Caster



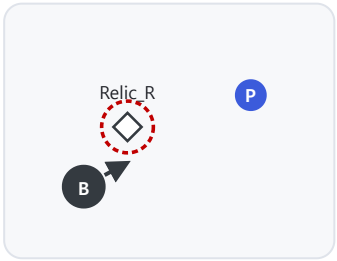
시전자 위치

Target



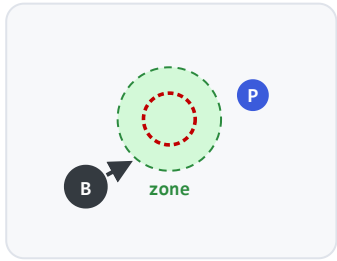
지금 타겟 (Fire 순간)

Socket



AnchorSet 키

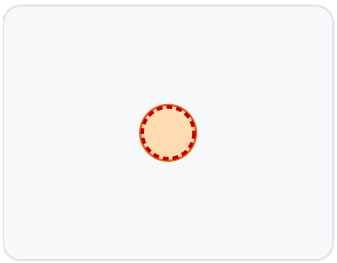
Saved



memo에 저장된 점

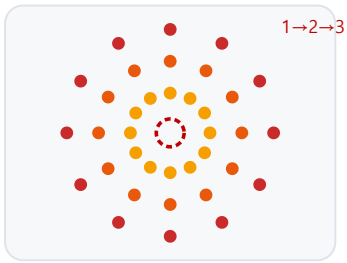
모양 (Placement 종류)

PointPlacement



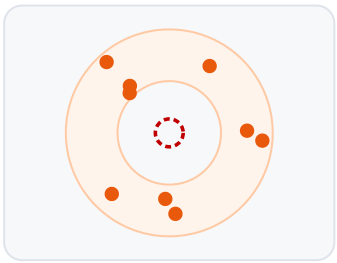
기준점 그 자리 1개

CirclePlacement



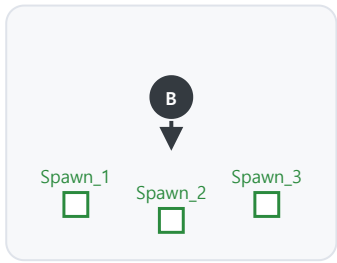
radii[] = 동심원, 순서대로

RandomInRingPlacement



minR ~ maxR 고리 안 무작위

SocketsPlacement



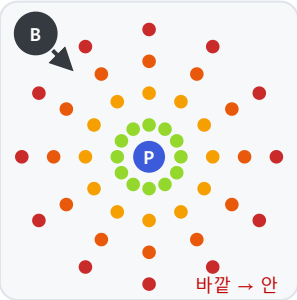
키 여러 개 = 소켓 여러 개

이번에 늘어난 것

1. **Anchor.Saved**
   - 앞 Event가 `saveAs`로 memo에 적은 점을 기준점으로 — 패턴 3의 "장판 중심 기준 안 / 밖"
2. **CirclePlacement.radii[]**
   - 반지름을 여러 개 주면 동심원, 넣은 순서 = 터지는 순서 (8 → 6 → 4 → 2 = 조여 옴)
3. **PointFacing (Inherit · Outward · Inward)**
   - 점 하나하나가 볼 쪽 — 기하라서 Placement가 정한다. 프리팹 자세 보정(`_vfxEuler`)만 Event에

5. 생각 중인 패턴 4개 — 전부 에셋 조립

1 조여오는 번개



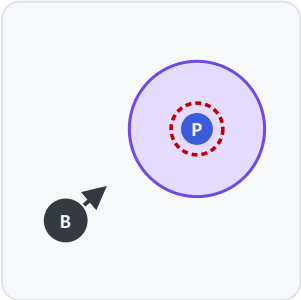
`Event.Strike`

StrikeEvent

- Circle · Target
- radii 8 / 6 / 4 / 2
- countPerRing 12
- interval 0.4
- batch 12

시전 순간 위치에 고정 —
Collect가 Fire 때 한 번

2 플레이어 위치에 장판



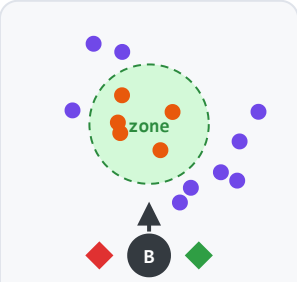
`Event.Strike`

FieldEvent

- Point · Target
- _field = 장판 프리팹
- _duration 5초

수명은 따로 띄운 대기 —
다음 점을 막지 않음,
사망 · 회수면 즉시 회수

3 유물 색에 따라 안 / 밖



relic 0 → 안 relic 1 → 밖

`Event.Cast`

- Choose(relic, 2)
- Activation(choiceKey relic)
- Field(Point·Caster 앞5m, saveAs zone)

`Event.Strike`

`if relic == 0`

- Strike · RandomInRing 0~r

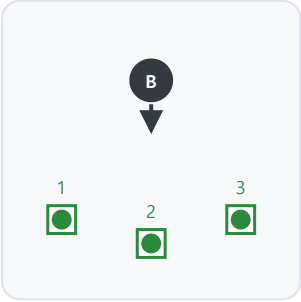
`if relic == 1`

- Strike · RandomInRing r~r+6

`Event.End`

- Activation(off)

4 지정 위치에 소환



`Event.Summon`

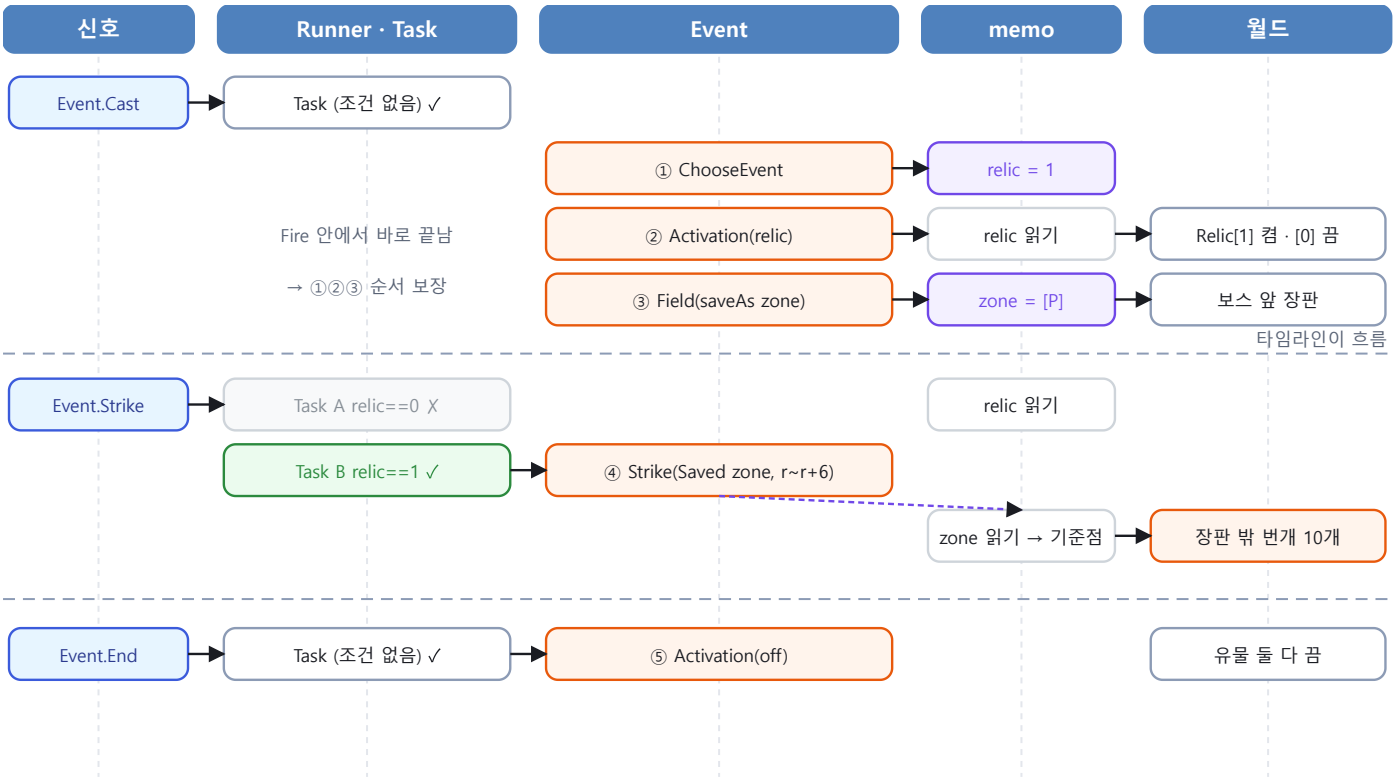
SummonEvent

- Sockets
- Spawn_1, Spawn_2, Spawn_3
- interval 0.3
- _monsterKey

로드 중 취소되면 소환수 회수,
ISummonRegistry 있으면 등록

새 코드 0줄 — Event 5개 · Placement 4개 조합과 Task 조건만으로 네 패턴이 나온다

6. 패턴 3 실행 순서 — memo와 조건이 어떻게 이어지나



분기가 Event 안에 없는 이유

1. 고르기와 분기를 뗐다

- ChooseEvent는 memo에 적기만, 분기는 Task 조건이 — StrikeEvent 안에는 if가 하나도 없다

2. 같은 Task 안 순서가 보장된다

- Choose · Activation · Field는 기다리는 게 없어 `Fire` 호출 안에서 끝남 → 뒤 Event가 memo를 바로 읽는다

3. memo는 패턴 하나 동안만

- Runner가 하나를 들고 패턴 시작마다 비운다 — 이전 패턴의 relic · zone이 남지 않는다

7. 바뀐 파일

파일	역할	상태
`SkillSystem/PatternMemo.cs`	패턴 동안의 칠판 — 저장한 점 · 고른 값. Runner가 재사용, 시작마다 비움	신규
`SkillSystem/Pattern.cs`	PatternContext에 `memo` 추가 (Runner가 채움)	변경
`SkillSystem/PatternRunner.cs`	memo 소유 · Start에서 비우고 ctx에 넣음 · OnEvent가 `Task.Matches`로 거름	변경
`SkillSystem/PatternTask.cs`	조건 `_ifKey / _ifValue` 추가 — 분기는 여기서	변경
`SkillSystem/AnchorSet.cs`	키 → Transform. 첫 질문 때 표를 만듦 (풀 첫 스폰은 Awake 전)	변경
`Pattern/Placement/Placement.cs`	Anchor(Caster · Target · Socket · **Saved**) · Resolve · Collect · 기즈모 미리보기	변경
`Pattern/Placement/*Placement.cs`	Point · Circle(radii[]) · RandomInRing · Sockets	신규 3 변경 1
`Pattern/Event/PatternEvent.cs`	공용 `SpawnAt` (풀 꺼내기 + Clear · Play)	변경
`Pattern/Event/PointEmitter.cs`	어디에 · 간격 · 묶음 · 저장 — 대기 뒤 토큰 확인, 리스트 풀 반납	신규
`Pattern/Event/*Event.cs`	Strike(VFX만) · Field · Summon · Activation · Choose	신규 3 변경 2

남은 일

1. **보스 프리팹 다시 연결 (에디터)**
   - 필드 구조가 바뀌어 StrikeEvent 2개 · CirclePlacement 1개 · SummonEvent 1개의 값이 비어 있음 → 다시 지정
   - AnchorSet 컴포넌트 추가 + 소켓 등록 (Relic · Spawn_1~3) · 태그 에셋 `Event.Cast` · `Event.End` 생성
2. **플레이 확인**
   - 지금은 컴파일만 확인 — 네 패턴을 조립해 실제로 돌려 봐야 한다
3. **피해**
   - StrikeEvent · FieldEvent는 지금 VFX · 장판만. 공용 HitEffect를 점 하나 = 판정 하나로 붙일 자리만 남겨 둠
4. **유물 정리**
   - 사망으로 끊기면 Event.End가 안 와 유물이 켜진 채 남음 → OnSpawn 리셋 또는 "취소 시 되돌림" 중 결정
5. **패턴 끝 vs 진행 중 Event**
   - 연속 타격 · 장판 수명은 PatternEnded 뒤에도 돈다 — 지금은 타임라인 길이로 맞추는 에셋 규칙
6. **이번 범위 밖**
   - WarningEvent · SavedPlacement(경고 → 타격 같은 자리) — 04 문서 7절 설계대로 다음에